

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación

Práctica de Laboratorio Número 1

Profesor:
José Canache
Arquitectura del Computador

Autor:
Ronald Montilla
C.I. 32.011.453

19 de julio de 2026

Índice

1. Código para los ejercicios del laboratorio:	2
1.1. Implementación recursiva:	2
1.2. Implementación iterativa:	3
2. Implementación de la recursividad en MIPS32 y el papel de la pila \$sp	3
2.1. Primera fase:	3
2.2. Segunda fase:	4
2.3. Tercera fase:	4
2.4. Cuarta fase:	4
3. Riesgos de desbordamientos y su mitigación	5
3.1. Desbordamiento de pila(stack overflow):	5
3.2. Desbordamiento aritmético(integer overflow):	5
4. Diferencias entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros	6
5. Diferencias entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32	7
6. Tutorial de la ejecución paso a paso en MARS	7
7. Justificación del enfoque según eficiencia y claridad en MIPS	8
8. Análisis y Discusión de los Resultados	9

1. Código para los ejercicios del laboratorio:

A continuación el código implementado para calcular la paridad de un número entero positivo(sin incluir el main del código...):

1.1. Implementación recursiva:

```
1      pariedadRecursivo: # Funcion recursiva que calcula la
      pariedad de un numero
2          # $a0 contendra el valor "n" cuya pariedad se determinara
      .
3      addi $sp, $sp, -12
4      sw $ra, 8($sp)
5      sw $a0, 4($sp)
6      slt $t0, $a0, $zero
7      bne $t0, $zero, errorRecursivo # Validacion de entrada,
      debe ser positiva.
8      beq $a0, $zero, casoBase      # caso base: el numero es
      igual a cero
9      li $v0, 1
10     sw $v0, 0($sp)
11     addi $a0, $a0, -1
12     jal pariedadRecursivo # Caso recursivo, se evaluo el
      numero menos una unidad.
13     move $t0, $v0
14     lw $v0, 0($sp)
15     sub $v0, $v0, $t0
16     j finPariedadRecursivo
17     errorRecursivo:
18         li $v0, -1
19         j finPariedadRecursivo
20     casoBase:
21         li $v0, 0
22     finPariedadRecursivo:
23         lw $a0, 4($sp)
24         lw $ra, 8($sp)
25         addi $sp, $sp, 12
26         jr $ra
```

El algoritmo consiste en cuatro partes(vease el apartado 2), y su funcionamiento es simple, si el número es cero se la función retornara 0, en cambio, si el número es mayor a cero se retornara 1 menos el resultado que se obtiene al llamar recursivamente a la función restando una unidad al número que se está evaluando, es decir el argumento:

visualmente sería: **return (1 - pariedadRecursivo(n - 1))**

De esta manera, si el resultado general, cuando finaliza todo el proceso recursivo, es cero significa que el número es positivo, si es uno significa que el número es negativo y si es -1 significa que el usuario ingreso un número invalido para ser evaluado por la función.

1.2. Implementación iterativa:

```
1  pariedad: # Funcion que calcula la pariedad de un numero.
2      addi $sp, $sp, 8
3      sw $ra, 4($sp)
4      sw $a0, 0($sp)
5      slt $t0, $a0, $zero
6      bne $t0, $zero, errorPariedad # Validacion de dato de
7                                     entrada, debe ser positivo.
8      li $v0, 0
9      for: # Ciclo que calcula la pariedad de un numero.
10         beq $a0, $zero, finPariedad
11         li $t0, 1
12         sub $v0, $t0, $v0
13         sub $a0, $a0, $t0
14         j for
15 errorPariedad:
16     li $v0, -1
17 finPariedad:
18     lw $a0, 0($sp)
19     lw $ra, 4($sp)
20     addi $sp, $sp, 8
21     jr $ra
```

Esta implementación es sumamente más sencilla, en primer lugar se valida el dato de entrada (el número cuya paridad será evaluada) si es cero se retornará cero, es un número negativo se retornará -1 para indicar que el dato ingresado es inválido en la función, pero, si es mayor a cero entrará directamente en una estructura repetitiva que se repetirá n veces, al final retornará un cero para indicar que el número es positivo, o uno para indicar que el número es negativo.

2. Implementación de la recursividad en MIPS32 y el papel de la pila \$sp

En MIPS32 la recursividad se basa en cuatro fases principales:

2.1. Primera fase:

La primera fase empieza con la llamada de la función recursiva, una vez realizada se emplea el stack pointer (\$sp) para guardar los argumentos, el registro de retorno (\$ra), y demás datos que deben ser guardados. Es acá donde aparece el papel de la pila, esta nos permitirá guardar los datos importantes de nuestro programa para que, al ser modificados, no se pierdan estos datos. En nuestro código de pariedadRecursivo:

```

1 pariedadRecursivo: # Funcion recursiva que calcula la pariedad de
    un numero
2     # $a0 contendra el valor "n" cuya pariedad se determinara
    .
3     addi $sp, $sp, -12
4     sw $ra, 8($sp)
5     sw $a0, 4($sp)

```

2.2. Segunda fase:

en la segunda fase se evalúa el caso base, es decir, el caso en donde la función finaliza. Si se cumple el caso base se debe saltar a la cuarta fase, ignorando la tercera fase (la recursiva). En la segunda fase, por lo general, se emplean instrucciones de comparación y salto condicional como `slt`, `beq`, `bne`, etc..., y por lo general el caso base suele modificar algún dato, como por ejemplo el registro de retorno `$v0` en funciones. En el código de `pariedadRecursivo`:

```

1     slt $t0, $a0, $zero
2     bne $t0, $zero, errorRecursivo # Validacion de entrada,
    debe ser positiva.
3     beq $a0, $zero, casoBase      # caso base: el numero es
    igual a cero

```

2.3. Tercera fase:

Corresponde a la fase recursiva, aquella en la que se modifican los argumentos principales para hacer el problema más pequeño. Mayormente el argumento se suele decrementar e inmediatamente se hace la llamada recursiva, luego se pasa a la primera, segunda y tercera fase hasta que el caso recursivo se cumpla y de esta manera liberar poco a poco el stack. En mi caso, también guardo el valor de `$v0`.

```

1     li $v0, 1
2     sw $v0, 0($sp)
3     addi $a0, $a0, -1
4     jal pariedadRecursivo # Caso recursivo, se evaluo el
    numero menos una unidad.

```

2.4. Cuarta fase:

Fase final cuyaa idea principal es recuperar los datos de la pila, restaurar el registro `$sp` y retornar un valor en caso de una función recursiva o modificar un registro en caso de un procedimiento recursivo.

```

1     move $t0, $v0
2     lw $v0, 0($sp)
3     sub $v0, $v0, $t0
4     j finPariedadRecursivo

```

```

1 finPariedadRecursivo:
2     lw $a0, 4($sp)
3     lw $ra, 8($sp)
4     addi $sp, $sp, 12
5     jr $ra

```

3. Riesgos de desbordamientos y su mitigación

En MIPS32 tener en consideración los riesgos de desbordamiento a la hora de crear un algoritmo es muy importante, ya que nos permitirá evitar errores y problemas con nuestro código a la hora de ser implementado. Existen dos riesgos principales de desbordamiento, el desbordamiento de pila y el aritmético.

3.1. Desbordamiento de pila(stack overflow):

La pila en MIPS32 es limitada, lo que quiere decir que se debe tratar con mucho cuidado, el desbordamiento de pila se produce cuando un programa resta tanto espacio a la pila $\$sp$ para guardar datos en ella, que el programa o procesador detectan que ya no hay más espacio porque pueden verse afectados los demás registros y se produce el desbordamiento de pila, en donde el programa puede finalizar de golpe, perder datos, etc.

En el código de “pariedadRecursivo” cada llamada a la función recursiva ocupa 12 bytes por llamada, ahora bien, si el usuario introduce un n muy grande puede llegar a producirse un desbordamiento de pila, ya que la función sera llamada n veces hasta llegar al caso base y por cada llamada la pila tendrá 12 bytes menos de espacio.

Ahora bien, ¿Como mitigar el desbordamiento de pila?, esto se hace principalmente validando un limite, es decir, colocar un limite del tamaño que puede tener n para que no se produzca un stack overflow con las llamadas de la función que consumen el espacio de la pila. De esta manera, no se producirá este problema pero la función se vera limitada a la hora de evaluar n 's muy grandes.

3.2. Desbordamiento aritmético(integer overflow):

MIPS32 tiene una característica muy importante, que la hace muy funcional pero limitada en ciertos aspectos, todo se basa en $32\ bits$, desde los registros, las instrucciones hasta las direcciones, esto es bueno a la hora de hablar de funcionalidad, eficiencia y simplicidad, pero trae consigo limitaciones claras, los $32\ bits$.

El desbordamiento aritmético se produce cuando al operar sobre dos registros su resultado pueda desbordarse, es decir, superar los 32 bits que limita MIPS32. Cuando pasa esto, se pueden producir muchos problemas, uno de los más comunes es que se produzca un resultado erróneo e inesperado ya que el desbordamiento suele alterar los bits más significativos.

Pero, ¿Como evitar o mitigar el desbordamiento aritmético?. Para evitar esto si estamos trabajando con positivos, lo ideal es operar sin signo, es decir, emplear instrucciones que operen sin signo(`addu`, `subu`, etc), de esta manera se tiene más margen sobre las operaciones aritméticas y es más fácil evitar este tipo de desbordamiento. Ahora bien, si se debe

operar con signos también se puede emplear el uso de instrucciones sin signo e inmediatamente después emplear unas líneas de código para verificar o validar los signos. Por otro lado, el programa(*MARS*) también interviene en la mitigación de este desbordamiento ya que al producirse detiene el programa de inmediato indicando que se está sobre los límites, esto es aceptado en muchos casos, pero en otros no, ya que detiene el programa.

En el caso del algoritmo recursivo de “pariedadRecursivo” este problema no va a ocurrir ya que los resultados estarán entre 0 y 1, por lo cual no se producirá un desbordamiento aritmético.

4. Diferencias entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros

Las implementaciones iterativa (*pariedad.asm*) y recursiva (*pariedadRecursivo.asm*) presentan diferencias en la gestión de los recursos del procesador:

- **Uso de Memoria (Complejidad Espacial):** La versión iterativa mantiene una complejidad espacial constante $\mathcal{O}(1)$. Aunque realiza una reserva inicial en la pila al invocar la función, el ciclo interno no consume memoria adicional sin importar qué tan grande sea el número n . Por el contrario, la versión recursiva presenta una complejidad espacial lineal $\mathcal{O}(n)$, consumiendo de forma acumulativa 12 bytes en la pila por cada decremento del valor de entrada hasta alcanzar el caso base.
- **Uso y Conservación de Registros:** En el enfoque recursivo, el procesador se ve obligado a realizar un uso intensivo de accesos a memoria a través de instrucciones **sw** (store word) y **lw** (load word) para preservar los registros de preservación obligatoria como **\$ra** y el argumento **\$a0**. En la solución iterativa, la persistencia de los datos se resuelve casi en su totalidad dentro del banco de registros utilizando temporales (**\$t0**), minimizando drásticamente el acceso a la memoria.

De esta manera podemos observar claramente las diferencias entre ambas implementaciones y darnos cuenta de algo: elegir entre una implementación recursiva y una iterativa es sumamente importante a la hora de tomar en cuenta el factor de memoria. Las implementaciones recursivas son más propensas a emplear un uso mayor de la memoria, esto lo vemos apenas inicia el código puesto a que su primera fase consiste en emplear la pila para guardar datos, y con su tendencia a ser llamada sucesivamente el uso de memoria aumenta gradualmente, representando en muchas ocasiones un potencial desbordamiento de pila.

En resumen, las diferencias son las siguientes: la implementación recursiva es importante porque son empleadas en temas específicos y complejos como los árboles, los grafos, y en la matemática para códigos más limpios, pero estas implementaciones son propensas a un uso mayor de memoria. Por otro lado, la implementación iterativa es mucho más sencilla de implementar, por lo general no consume mucha memoria pero si emplea un mayor uso de los registros.

5. Diferencias entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32

Al comparar los ejemplos teóricos del libro de *Patterson y Hennessy* con las prácticas completas desarrolladas, se pueden notar diferencias bastante claras en cómo se estructura el código:

- **Falta de interacción con el usuario:** En los ejemplos del libro de Patterson y Hennessy, por lo general se muestran funciones o procedimientos aislados (como el cálculo de un factorial o el ordenamiento de un arreglo) asumiendo que los datos ya están cargados en los registros. En cambio, en un ejercicio real es obligatorio construir una sección `.data` para almacenar las cadenas de texto correspondientes a los mensajes del sistema, como por ejemplo: (`men1`, `men2`, `men3`, `men4`).
- **Uso del entorno y llamadas al sistema:** Los ejemplos académicos del libro rara vez incluyen el uso de `syscall` para interactuar con la consola de MARS. En un programa real, se necesita gestionar el registro `$v0` para pedir un número entero (código 5) o imprimir texto en pantalla (código 4).
- **Validación de datos de entrada:** Los ejemplos del libro suelen asumir que los datos de entrada siempre van a ser perfectos y válidos. En nuestros códigos, se tuvo que añadir instrucciones explícitas de comparación y salto condicional (`slt` y `bne`) para verificar si el usuario introdujo un número negativo y así evitar que el programa falle o entre en un ciclo infinito, redirigiendo el flujo a una rutina de error.
- **Finalización formal del programa:** En la teoría del libro las funciones simplemente terminan, pero en un entorno de ejecución real como MARS, si no se añade de forma obligatoria las líneas `li $v0, 10` seguidas de un `syscall` al terminar el bloque `main`, el procesador sigue ejecutando de largo las siguientes líneas de código, lo cual causaría un error de ejecución.

En resumen, las principales diferencias están en que los ejercicios completos demandan mayor lógica del programador ya que debe tener en cuenta muchos factores a la hora de crear una función, como por ejemplo la validación de datos. Los ejercicios del libro solo se centran en realizar la función más no toman en cuenta la iteración con el usuario y la consola de MARS.

6. Tutorial de la ejecución paso a paso en MARS

Para poder observar con detalle cómo trabaja el procesador con nuestros programas, el simulador MARS nos ofrece la herramienta de ejecución paso a paso. El proceso se realiza mediante las siguientes instrucciones:

Paso 1: Carga y edición del archivo: Lo primero que debemos hacer es abrir el entorno MARS y abrir el archivo con extensión `.asm` que queramos probar (ya sea `pariedad.asm` o `pariedadRecursivo.asm`) desde la pestaña *Edit*.

Paso 2: Ensamblado del código: Una vez abierto el código, hacemos clic en el botón con el ícono de una herramienta y un destornillador (*Assemble*). Si no hay errores de sintaxis, MARS cambiará automáticamente de la pestaña *Edit* a la pestaña *Execute*.

En esta pantalla veremos cómo nuestro código se traduce a direcciones de memoria y lenguaje de máquina.

Paso 3: Ejecución instrucción por instrucción: En lugar de presionar el botón de play (que corre todo el programa de golpe), utilizaremos el botón que tiene una flecha verde hacia la derecha acompañada de un número 1 (*Run one step*). Cada vez que presionemos este botón, el simulador ejecutará una única línea de código, resaltando en color amarillo la instrucción que se va a procesar a continuación.

Paso 4: Monitoreo de registros y consola: A medida que avanzamos paso a paso, debemos prestar atención a la parte derecha de la pantalla, donde se encuentra el banco de registros. Allí veremos cómo cambian los valores de `$a0`, `$v0` y, sobre todo, cómo se modifica la dirección del `$sp` al guardar datos en la pila. Al mismo tiempo, en la parte inferior (pestaña *Run I/O*) aparecerán las solicitudes de texto para introducir el número entero y los mensajes finales de “Es par” o “Es impar” según corresponda.

7. Justificación del enfoque según eficiencia y claridad en MIPS

Al evaluar las dos soluciones implementadas para determinar la paridad de un número entero positivo, se pueden analizar bajo dos criterios clave:

Desde el punto de vista de la eficiencia: El enfoque iterativo es considerablemente más eficiente en MIPS32 que el enfoque recursivo. En el código iterativo de `pariedad.asm`, el bucle `for` resuelve todo el cálculo modificando directamente los registros temporales y restándole unidades al valor de entrada. En cambio, la versión recursiva realiza una manipulación constante de la memoria RAM, restando 12 bytes a la pila por cada llamada recursiva para resguardar los registros `$ra` y `$a0`. Esta acumulación de accesos a memoria mediante instrucciones `sw` y `lw` ralentiza el rendimiento del procesador y consume memoria de forma lineal innecesaria.

Desde el punto de vista de la claridad: Generalmente, la recursividad suele verse más limpia en el papel porque copia la definición original del problema. Sin embargo, al pasarlo a lenguaje ensamblador MIPS32, la claridad se pierde bastante debido a que el programador debe encargarse manualmente de todo el protocolo de la pila (guardar, restar espacio, recuperar y sumar espacio). Esto hace que el código recursivo sea más largo y propenso a errores de lógica. Por su parte, la estructura del ciclo iterativo resulta mucho más directa de leer línea por línea para cualquier persona que analice el código ensamblador.

Por lo tanto, la elección ideal para este problema específico es el enfoque iterativo, ya que ofrece un rendimiento superior y un código mucho más simple de mantener en este lenguaje.

8. Análisis y Discusión de los Resultados

Luego de completar y ejecutar de forma satisfactoria ambos programas en el simulador MARS, se pudieron comprobar de manera práctica los conceptos teóricos de la arquitectura:

En primer lugar, al ingresar valores de prueba pequeños (como por ejemplo el número 4 o el 7), ambas implementaciones arrojaron los mensajes esperados en la consola de manera inmediata, confirmando que la lógica de ambos algoritmos es correcta a la hora de discernir si el número ingresado es par o impar. De igual forma, las rutinas de validación implementadas cumplieron su rol deteniendo el flujo normal del programa y mostrando el mensaje de error correspondiente cuando se introducían números que no formaban parte de los enteros positivos válidos.

Sin embargo, la discusión real aparece al evaluar el comportamiento interno de la memoria. Mediante el seguimiento paso a paso, se evidenció que la versión recursiva es sumamente frágil ante entradas de datos medianamente altas. El tener que descontar espacio en la pila de manera repetitiva por cada decremento expone de forma directa al sistema a fallas críticas de desbordamiento, un comportamiento que no ocurre con la versión iterativa, cuyo uso de la memoria se mantiene completamente estático a lo largo de toda su ejecución en el bucle.

Esto nos permite concluir que, en el desarrollo sobre lenguaje ensamblador, la elegancia de una solución algorítmica siempre debe balancearse con las limitaciones físicas del hardware en el que va a operar.